Kea2 性质检查模块技术报告

1 技术原理复盘

1.1 模块目标与解决的问题

Kea2 的性质检查模块旨在解决 Android 应用非崩溃功能 bug（占 APP 总 bug 的 65.4%）难以自动化检测的问题。该模块通过**基于性质的测试**（Property-based Testing, PBT），在 APP 动态执行过程中自动验证用户指定的程序性质，精准识别功能异常。

核心解决的问题：

- **测试预言缺失**：传统自动化 GUI 测试缺乏有效机制判断功能是否正确。
- **手动测试成本高**：功能 bug 检测依赖人工验证，效率低下。
- **覆盖不足**：现有工具仅能覆盖 21% 的功能 bug，且输入质量低。

1.2 输入与输出

输入：

1. **性质规约**：用户通过 PDL（Property Description Language，基于 Python 的领域特定语言）编写的性质定义，包含：
    - **前置条件**（Precondition, P）：使用 `@precondition` 装饰器定义，指定性质可执行时的 GUI 状态。
    - **交互场景**（Interaction, I）：性质测试方法体中的操作序列（如点击按钮、输入文本、旋转屏幕等）。
    - **后置条件/断言**（Postcondition/Assertion, Q）：在测试方法中使用 `assert` 语句定义，验证交互后的程序状态。
2. **待测 APK 文件**：目标 Android 应用的安装包。
3. **探索策略配置**：随机探索或主路径引导探索的参数设置。

输出：

1. **性质检查结果**：通过（pass）/违反（fail/error）。
2. Bug **报告**：违反性质时生成 HTML 格式的详细报告，包含：
   - 触发 bug 的 GUI 测试用例（事件轨迹）。
   - 程序状态快照（截图）。
   - 性质执行统计信息。
   - 堆栈轨迹（traceback）。

1.3 基本流程与代码架构

1.3.1 基本流程（三阶段）

阶段 (a)：获取用户定义的性质作为输入

```
# keaUtils.py:419-422
def run(self, test):
    self.allProperties = dict()
    self.collectAllProperties(test)  # 收集所有标记了@precondition的测试方法
```

- **入口**：`KeaTestRunner.run()` 方法。
- **机制**：通过反射遍历所有 `unittest.TestCase`，查找标记了 `PRECONDITIONS_MARKER`（即 `@precondition` 装饰器）的测试方法。
- **存储**：将性质存储到 `self.allProperties` 字典中，键为方法名，值为 `TestCase` 实例。

阶段 (b)：当满足前置条件时，在动态执行过程中暂停，将控制权转移以执行性质检查

这是模块的核心机制，具体实现如下：

```
# keaUtils.py:474-544
while self.stepsCount < self.options.maxStep:
    # 1. 获取当前GUI状态（通过Fastbot发送Monkey事件或直接dump hierarchy）
    if fb.executed_prop:
        # 如果刚执行完性质，直接获取当前状态，不发送新事件
        xml_raw = fb.dumpHierarchy()
    else:
        # 正常探索：发送Monkey事件并获取GUI状态
        self.stepsCount += 1
        xml_raw = fb.stepMonkey(self._monkeyStepInfo)

    # 2. 检查所有性质的前置条件（暂停探索，进行状态检查）
    propsSatisfiedPrecond = self.getValidProperties(xml_raw, result)
```

```

    # 3. 如果没有满足前置条件的性质，继续探索（控制权返回给探索循环）
    if len(propsSatisfiedPrecond) == 0:
        continue

    # 4. 有满足前置条件的性质时，暂停探索，转移控制权执行性质检查
    # ... 概率过滤和选择性质 ...
    execPropName = random.choice(propsNameFilteredByP)
    test = propsSatisfiedPrecond[execPropName]

    # 5. 执行性质测试方法（控制权转移到用户定义的性质代码）
    test(result)  # 执行交互场景I和断言Q

    # 6. 性质执行完成后，标记已执行，下次循环直接获取状态
    fb.executed_prop = True
```

暂停机制的关键点：

- **暂停探索**：当检测到满足前置条件的性质时，不再继续发送 Monkey 事件（`stepsCount` 不增加）。
- **控制权转移**：从 Fastbot 的探索循环转移到性质测试方法的执行（`test(result)` 调用）。
- **状态保持**：通过 `fb.executed_prop` 标志，确保性质执行后能获取当前 GUI 状态，而不是立即发送新事件。

阶段 (c)：根据用户定义的" 前置条件-后置条件 (断言)" 进行性质检查，并将结果记录为输出

```
# keaUtils.py:536-544
try:
    test(result)  # 执行性质测试方法
    # 方法体内包含：
    # - 交互场景I：self.d(text="Settings").click() 等操作
    # - 后置条件Q：assert self.d(...).exists() 等断言
finally:
    result.printError(test)

# 记录执行结果
result.updateExectedInfo()  # 更新状态为pass/fail/error
fb.logScript(result.lastExecutedInfo)  # 记录到Fastbot日志
result.flushResult()  # 持久化到JSON文件
```

- **断言评估**：性质测试方法中的 `assert` 语句如果失败，会抛出 `AssertionError`。

- **异常捕获**:`unittest.TestCase` 的 `__call__` 方法捕获异常,调用 `result.addFailure()` 或 `result.addError()`。
- **结果记录**:执行信息(开始步骤数、状态、堆栈轨迹)记录到 `PropertyExecutionInfo`,并持久化到文件。

1.3.2 代码架构

主要类/函数分工:

1. PDL **解析器**(`keaUtils.py` 中的装饰器系统):
   - `@precondition`(`keaUtils.py:54-65`):解析前置条件,支持多个前置条件(逻辑 AND)。
   - `@prob`(`keaUtils.py:68-81`):定义性质执行概率(0-1 之间)。
   - `@max_tries`(`keaUtils.py:84-97`):限制性质最大执行次数。
   - `collectAllProperties()`(`keaUtils.py:631-672`):通过反射收集所有标记了 `@precondition` 的测试方法。
2. **探索策略模块**(`fastbotManager.py` + `keaUtils.py`):
   - `FastbotManager`:管理 Fastbot 服务,发送 Monkey 事件,获取 GUI 状态。
   - `KeaTestRunner.run()`:主循环,协调探索和性质检查。
3. **状态收集器**(`u2Driver.py`):
   - `U2StaticChecker`:基于 XML hierarchy 进行静态检查(用于前置条件评估)。
   - `U2ScriptDriver`:可执行实际操作的驱动(用于性质执行时的交互)。
4. **断言评估器**(`keaUtils.py` 中的 `KeaTestRunner`):
   - `JsonResult.addFailure/addError()`:捕获断言失败和异常,记录执行信息。
5. **结果记录与报告生成**(`keaUtils.py` + `report/bug_report_generator.py`):
   - `JsonResult`:实时记录性质执行统计。
   - `BugReportGenerator`:生成 HTML 格式的 bug 报告。

2 源码阅读与逻辑拆解

2.1 完整逻辑链:从输入到输出

2.1.1 输入阶段:性质定义与收集

入口函数:`kea_launcher.py:run()` → `KeaTestRunner.run()`

性质收集流程 (keaUtils.py:631-672):

```python
def collectAllProperties(self, test: TestSuite):
    def iter_tests(suite):
        for test in suite:
            if isinstance(test, TestSuite):
                yield from iter_tests(test)
            else:
                yield test

    # 遍历所有TestCase，查找标记了@precondition的测试方法
    for t in iter_tests(test):
        testMethodName = t._testMethodName
        testMethod = getattr(t, testMethodName)

        # 检查是否有PRECONDITIONS_MARKER标记（即@precondition装饰器）
        if hasattr(testMethod, PRECONDITIONS_MARKER):
            # 移除setUp/tearDown钩子（PBT不需要）
            remove_setUp(t)
            remove_tearDown(t)
            # 保存到allProperties字典
            self.allProperties[testMethodName] = t
```

性质定义示例 (properties/Omninotes_Sample.py):

```python
@prob(0.7)  # 执行概率70%
@precondition(
    lambda self: self.d(text="Omni Notes Alpha").exists
    and self.d(text="Settings").exists
)
def test_goToPrivacy(self):
    """交互场景：点击Settings -> Privacy"""
    self.d(text="Settings").click()
    sleep(0.5)
    self.d(text="Privacy").click()

@max_tries(1)  # 最多执行1次
@precondition(
    lambda self: self.d(resourceId="it.feio.android.omninotes.alpha:id/search_src_text").exists
)
def test_rotation(self):
    """交互场景：旋转设备，断言：搜索框应仍然存在"""
    self.d.set_orientation("l")
    sleep(2)
```

```
    self.d.set_orientation("n")
    sleep(2)
    # 后置条件/断言Q
    assert self.d(resourceId="it.feio.android.omninotes.alpha:id/search_src_text").exists()
```

2.1.2 处理阶段：探索循环与前置条件检查

主探索循环（keaUtils.py:474-544）：

```
while self.stepsCount < self.options.maxStep:
    # === 步骤1：获取GUI状态 ===
    if fb.executed_prop:
        # 刚执行完性质，直接获取当前状态（不发送新事件）
        fb.executed_prop = False
        xml_raw = fb.dumpHierarchy()  # 通过HTTP请求获取XML hierarchy
    else:
        # 正常探索：发送Monkey事件
        self.stepsCount += 1
        xml_raw = fb.stepMonkey(self._monkeyStepInfo)  # 发送事件并获取GUI状态

    # === 步骤2：检查所有性质的前置条件（暂停探索） ===
    propsSatisfiedPrecond = self.getValidProperties(xml_raw, result)

    # === 步骤3：如果没有满足前置条件的性质，继续探索 ===
    if len(propsSatisfiedPrecond) == 0:
        continue

    # === 步骤4：概率过滤 ===
    p = random.random()
    propsNameFilteredByP = []
    for propName, test in propsSatisfiedPrecond.items():
        result.addPrecondSatisfied(test)  # 记录前置条件满足次数
        if getattr(test, PROB_MARKER, 1) >= p:
            propsNameFilteredByP.append(propName)

    if len(propsNameFilteredByP) == 0:
        continue

    # === 步骤5：随机选择一个性质执行（控制权转移） ===
    execPropName = random.choice(propsNameFilteredByP)
    test = propsSatisfiedPrecond[execPropName]

```

```python
    # 依赖注入：将可执行操作的driver注入到test实例
    self.scriptDriver = U2Driver.getScriptDriver(mode="proxy")
    setattr(test, self.options.driverName, self.scriptDriver)

    # === 步骤6：执行性质测试方法 ===
    result.addExcuted(test, self.stepsCount)  # 记录执行开始
    fb.logScript(result.lastExecutedInfo)
    try:
        test(result)  # 执行交互场景I和断言Q
    finally:
        result.printError(test)

    # === 步骤7：记录结果 ===
    result.updateExectedInfo()  # 更新状态（pass/fail/error）
    fb.logScript(result.lastExecutedInfo)
    fb.executed_prop = True  # 标记已执行性质
    result.flushResult()  # 持久化结果到JSON文件
```

前置条件检查实现（keaUtils.py:589-629）：

```python
def getValidProperties(self, xml_raw: str, result: JsonResult) -> PropertyStore:
    # 创建静态检查器（基于XML hierarchy，不连接设备，性能高）
    staticCheckerDriver = U2Driver.getStaticChecker(hierarchy=xml_raw)

    validProps: PropertyStore = dict()
    for propName, test in self.allProperties.items():
        valid = True
        prop = getattr(test, propName)  # 获取测试方法对象

        # 检查所有前置条件（逻辑AND，所有precond都必须返回True）
        for precond in prop.preconds:  # preconds是@precondition装饰器添加的元组
            # 依赖注入：将静态检查器注入到test实例（用于precond lambda中访问self.d）
            setattr(test, self.options.driverName, staticCheckerDriver)
            try:
                # 执行前置条件lambda函数：precond(test)
                # 例如：lambda self: self.d(text="Settings").exists
                if not precond(test):
                    valid = False
                    break
            except u2.UiObjectNotFoundError:
                valid = False
```

```
                break
            except Exception as e:
                logger.error(f"Error when checking precond: {getFullPropName(test)}")
                valid = False
                break

        # 如果所有前置条件都满足，检查是否达到最大执行次数
        if valid:
            if result.getExcuted(test) >= getattr(prop, MAX_TRIES_MARKER, float("inf")):
                continue
            validProps[propName] = test

    staticCheckerDriver.clear_cache()  # 清理缓存
    return validProps
```

状态收集实现 (u2Driver.py:515-543):

```
class U2StaticChecker:
    def __init__(self):
        self.d = U2StaticDevice(U2ScriptDriver().getInstance())

    def setHierarchy(self, hierarchy: str):
        # 解析XML hierarchy为etree.Element
        self.d.xml = etree.fromstring(hierarchy.encode("utf-8"))
        # 过滤被覆盖的控件 (hidden widget filter)
        _HindenWidgetFilter(self.d.xml)

    def getInstance(self, hierarchy: str=None):
        self.setHierarchy(hierarchy)
        return self.d  # 返回U2StaticDevice实例
```

控件存在性检查 (u2Driver.py:192-197):

```
@property
def exists(self):
    set_covered_to_deepest_node(self.selector)  # 设置covered属性
    xpath = self.selector_to_xpath(self.selector)  # 转换U2 Selector为XPath
    matched_widgets = self.session.xml.xpath(xpath)  # 在XML中查找
    return bool(matched_widgets)
```

2.1.3 输出阶段：性质执行与断言评估

性质执行机制（keaUtils.py:536-544）：

```
# 依赖注入：将脚本驱动（可执行操作的driver）注入到test实例
self.scriptDriver = U2Driver.getScriptDriver(mode="proxy")
setattr(test, self.options.driverName, self.scriptDriver)

# 记录执行开始
result.addExcuted(test, self.stepsCount)
fb.logScript(result.lastExecutedInfo)

try:
    # 执行性质测试方法（控制权转移到用户代码）
    test(result)  # 调用unittest.TestCase.__call__()
    # 方法体内执行：
    # 1. 交互场景I：self.d(text="Settings").click() 等操作
    # 2. 后置条件Q：assert self.d(...).exists() 等断言
finally:
    result.printError(test)  # 如果有错误，打印堆栈轨迹

# 更新执行信息（pass/fail/error）
result.updateExectedInfo()
fb.logScript(result.lastExecutedInfo)
fb.executed_prop = True  # 标记已执行性质，下次循环直接dump hierarchy
result.flushResult()  # 持久化结果到JSON文件
```

断言评估机制：

- 性质测试方法中的 assert 语句如果失败，会抛出 AssertionError。
- unittest.TestCase 的 __call__ 方法会捕获异常，调用 result.addFailure()。
- JsonResult.addFailure()（keaUtils.py:350-354）记录失败状态和堆栈轨迹：

```
def addFailure(self, test, err):
    super().addFailure(test, err)
    self.res[getFullPropName(test)].fail += 1
    self.lastExecutedInfo.state = "fail"
    self.lastExecutedInfo.tb = self._exc_info_to_string(err, test)
```

结果持久化（keaUtils.py:324-335）：

```
def flushResult(self):
    global RESFILE, PROP_EXEC_RESFILE
    # 持久化统计信息到result_*.json
    json_res = dict()
```

```
    for propName, propStatitic in self.res.items():
        json_res[propName] = asdict(propStatitic)
    with open(RESFILE, "w", encoding="utf-8") as fp:
        json.dump(json_res, fp, indent=4)

    # 持久化执行信息到property_exec_info_*.json（每行一个JSON对象）
    while self.executionInfoStore:
        execInfo = self.executionInfoStore.popleft()
        with open(PROP_EXEC_RESFILE, "a", encoding="utf-8") as fp:
            fp.write(f"{json.dumps(asdict(execInfo))}\n")
```

2.2 模块执行流程图

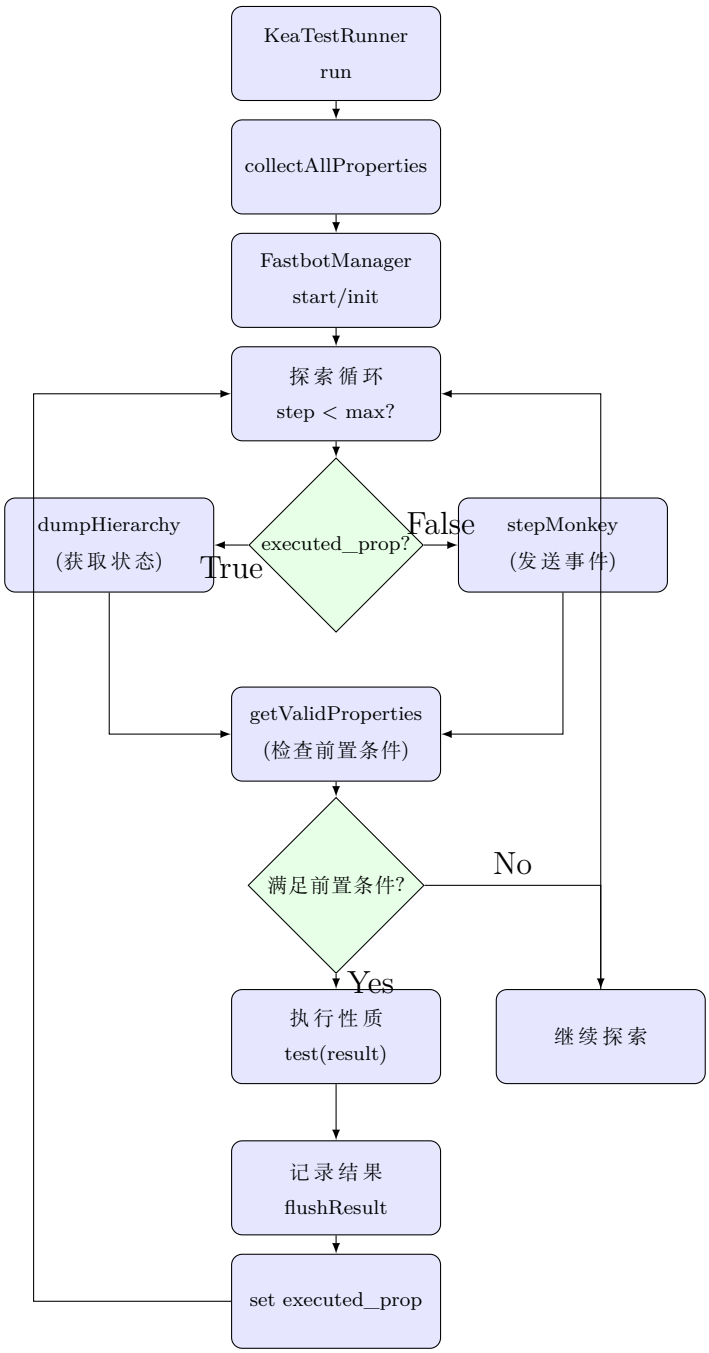

图 1: Kea2 模块执行流程图

关键函数调用关系标注:

- **性质收集**:`collectAllProperties()` → 反射遍历 TestCase → 检查 `PRECONDITIONS_MARKER`。
- **状态获取**:`FastbotManager.stepMonkey()` → HTTP 请求 → 返回 XML hierarchy。
- **前置条件检查**:`getValidProperties()` → `U2Driver.getStaticChecker()` → `U2StaticChecker` → `StaticU2UiObject.exists` → XPath 查询。

- **性质执行**：`test(result)` → `unittest.TestCase.__call__()` → 执行方法体 → 捕获异常 → `result.addFailure/addError()`。
- **结果持久化**：`result.flushResult()` → 写入 `result_*.json` 和 `property_exec_info_*.json`。

数据流转路径标注：

- **性质对象**：用户文件 → `allProperties` 字典 → `validProps` 字典 → `test` 实例 → `PropertyExecutionInfo` → JSON 文件。
- GUI **状态**：Fastbot → XML 字符串 → `etree.Element` → XPath 查询 → 布尔值 → 前置条件判断。
- **执行结果**：`assert` 失败 → `AssertionError` → `result.addFailure()` → `PropertyExecutionInf` → JSON 文件 → HTML 报告。

2.3 关键函数/方法调用关系详细说明

2.3.1 性质检查核心调用链

```
KeaTestRunner.run()
  |- collectAllProperties(test)
  |   `- iter_tests(suite)  # 递归遍历 TestSuite
  |       `- hasattr(testMethod, PRECONDITIONS_MARKER)  # 检查 @precondition 标记
  |           `- self.allProperties[testMethodName] = t  # 存储性质
  |
  |- FastbotManager.start()  # 启动 Fastbot 服务 (Android 端)
  |- FastbotManager.init()   # 初始化配置
  |
  `- while 循环 (探索循环) :
      |- FastbotManager.stepMonkey()  # 发送 Monkey 事件
      |   `- HTTP POST /stepMonkey -> 返回 XML hierarchy 字符串
      |
      |- getValidProperties(xml_raw, result)
      |   |- U2Driver.getStaticChecker(hierarchy=xml_raw)
      |   |   `- U2StaticChecker.setHierarchy(hierarchy)
      |   |       |- etree.fromstring()  # 解析 XML
      |   |       `- _HindenWidgetFilter()  # 过滤被覆盖控件
      |   |
      |   `- 遍历 allProperties, 执行 precond(test)
      |       |- setattr(test, driverName, staticCheckerDriver)  # 依赖注入
      |       `- precond(test)  # 执行 lambda 函数
      |           `- self.d(...).exists
      |               `- StaticU2UiObject.exists
      |                   |- selector_to_xpath()  # 转换 Selector 为 XPath
```

```
|                  `- session.xml.xpath(xpath)  # 查询 XML
|
|- 概率过滤 + 随机选择性质
|
|- U2Driver.getScriptDriver(mode="proxy")  # 获取可执行操作的 driver
|- setattr(test, driverName, scriptDriver)  # 依赖注入
|
|- test(result)  # 执行性质测试方法
|   |- unittest.TestCase.__call__(result)
|   |   |- 执行方法体 (交互场景 I)
|   |   |   `- self.d.click() 等操作 (U2ScriptDriver)
|   |   |
|   |   `- assert 语句 (后置条件 Q)
|   |       `- 如果失败 -> AssertionError
|   |
|   `- 捕获异常 -> result.addFailure/addError()
|
|- JsonResult.addFailure/addError()
|   `- 更新 PropertyExecutionInfo (state, tb)
|
`- result.flushResult()
    |- 写入 result_*.json (统计信息)
    `- 写入 property_exec_info_*.json (执行信息)
```

2.3.2 数据流转路径详细说明

性质对象流转:

```
用户编写的性质文件 (properties/*.py)
  (unittest 加载)
TestCase 实例 (标记 @precondition 装饰器)
  (collectAllProperties 收集)
allProperties 字典 {propName: TestCase}
  (getValidProperties 过滤)
validProps 字典 {propName: TestCase}
  (概率过滤 + 随机选择)
execPropName, test (选中的性质)
  (执行)
test(result) 调用
  (记录)
PropertyExecutionInfo (startStepsCount, propName, state, tb)
  (持久化)
result.json + property_exec_info_*.json
```

```
   (报告生成)
BugReportGenerator 解析  HTML 报告
```

GUI **状态流转**:

```
Fastbot (Android 端, 通过 Accessibility Service)
   (stepMonkey HTTP 请求)
XML hierarchy 字符串 (UI 控件树)
   (U2StaticChecker.setHierarchy)
etree.Element (解析后的 XML 树)
   (_HindenWidgetFilter)
过滤后的 XML 树 (标记 covered 属性)
   (StaticU2UiObject.exists 查询)
U2 Selector  XPath 转换
   (xpath 查询)
匹配的 XML 节点列表
   (bool 转换)
exists 属性值 (True/False)
   (precond lambda 返回)
前置条件满足判断 (True/False)
```

3 前沿技术关联

3.1 动态程序分析（Dynamic Program Analysis）

Kea2 的性质检查模块是动态程序分析的典型应用:

- **运行时监控**: 在 APP 实际执行过程中收集程序状态 (GUI 层次结构、控件属性)。
- **状态快照**: 通过 `dumpHierarchy()` 获取执行点的完整 GUI 状态 (XML hierarchy)。
- **执行轨迹记录**: 记录 Monkey 事件序列和性质执行信息，用于 bug 复现。

3.2 运行时验证（Runtime Verification）

该模块实现了轻量级的运行时验证框架:

- **性质规约语言**: PDL（基于 Python 装饰器）提供灵活的性质定义机制。
- **在线监控**: 在 APP 执行过程中实时检查性质，而非离线分析。
- **违反检测**: 通过断言评估自动检测性质违反，无需人工介入。

3.3 程序插装（Program Instrumentation）

虽然 Kea2 主要使用外部工具（Uiautomator2）获取状态，但其架构体现了插装思想：

- **代理模式**：通过 Fastbot 代理（port 8090）拦截和控制 APP 执行。
- **事件注入**：Monkey 事件可视为对 APP 的" 插装"，驱动执行路径。
- **状态提取**：通过 Android Accessibility Service 获取 GUI 状态，类似于插装点。

3.4 基于性质的测试（Property-based Testing）

这是该模块的核心技术范式：

- **性质驱动**：测试用例由性质规约（P→I→Q）驱动，而非固定输入。
- **自动生成输入**：通过探索策略自动生成 GUI 事件序列。
- **预言机制**：后置条件/断言作为测试预言，判断功能正确性。

3.5 符号执行（部分关联）

虽然 Kea2 主要使用具体执行（concrete execution），但其" 主路径引导探索" 策略体现了符号执行的启发：

- **路径推导**：基于性质的前置条件推导关键执行路径。
- **目标导向**：优先探索可能触发目标性质的路径，而非盲目随机。

3.6 混合测试（Hybrid Testing）

Kea2 支持传统单元测试与性质测试的混合执行：

- HybridTestRunner：允许在 unittest 中调用 `kea2_breakpoint()` 切换到性质测试。
- **无缝切换**：同一测试框架下支持两种测试范式，提升测试灵活性。

4 总结

Kea2 的性质检查模块通过 PDL 语言定义性质、动态探索驱动执行、实时状态收集与断言评估，实现了 Android 应用功能 bug 的自动化检测。其核心创新在于将" 用户指定性质" 与" 高效探索策略" 结合，解决了传统工具" 测试预言缺失" 和" 覆盖不足" 的问题。

关键技术机制：

1. **暂停执行与控制权转移**：当检测到满足前置条件的性质时，暂停 Fastbot 的探索循环，将控制权转移到性质测试方法的执行。
2. **依赖注入**：通过 `setattr` 动态注入不同的 driver（静态检查器用于前置条件，脚本驱动用于性质执行）。
3. **状态快照**：通过 XML hierarchy 获取完整的 GUI 状态，支持高效的前置条件检查。

该模块在 97 个历史 bug 中检测率达到 94.8%，并在实际应用中发现了 25 个新 bug，验证了其有效性。